

Documento de Arquitetura do Software Cervejaria BeboSim

Nilmar Pereira Sousa¹

¹Instituto Federal de Educação, Ciência e Tecnologia da Bahia (IFBA)
Campus Vitória da Conquista

nilmar.pereira14@gmail.com

Abstract. *This article presents the architectural definition of a web system called Cervejaria BeboSim. This document was created from the software requirements document of the referred system. The adopted architectural decisions and their consequences are discussed, having been adopted, as the format for recording such decisions, Architecture Decision Records (ADRs).*

Resumo. *Este artigo apresenta a definição arquitetural de um sistema web chamado, Cervejaria BeboSim. Este documento foi criado a partir do documento de requisitos de software do referido sistema. São discutidas as decisões arquiteturais adotadas e suas consequências, tendo sido adotado, como formato de registro de tais decisões, Architecture Decisions Records (ADRs).*

1. Introdução

Este documento apresenta a definição arquitetural do sistema Cervejaria BeboSim, um sistema *web* voltado à gestão das operações da cervejaria BeboSim, incluindo o controle de funcionários, clientes, produtos, pedidos, embalagens, equipes de vendas, campanhas publicitárias e unidades de produção. A arquitetura é derivada a partir dos requisitos do sistema, considerando as restrições de usabilidade, desempenho e manutenibilidade estabelecidas no Documento de Requisitos de Software.

2. Resumo dos Requisitos

Os principais Requisitos Funcionais (RF) do sistema incluem:

- **RF005–RF008:** Gerenciar Usuários – criação e controle de usuários do sistema vinculados a funcionários, com definição de perfis de acesso e senha;
- **RF009–RF012:** Gerenciar Clientes – cadastro e manutenção dos clientes para os quais os produtos são vendidos;
- **RF013–RF016:** Gerenciar Produtos – cadastro e manutenção dos produtos fabricados, incluindo fórmula de produção, preço e controle de estoque;
- **RF021–RF024:** Gerenciar Pedidos – registro de pedidos de venda com dependências obrigatórias de produto, vendedor e cliente;
- **RF029–RF032:** Gerenciar Embalagens – controle de embalagens vinculadas a produtos, com dados de custo, volume e tipo;
- **RF033–RF036:** Gerenciar Campanhas Publicitárias – registro de campanhas com dados financeiros, datas e percentual de retorno sobre vendas;
- **RF037:** Gerar Relatórios – geração de relatórios gerenciais de pedidos, vendas, clientes e campanhas por período.

Requisitos Não-Funcionais relevantes:

- **NF001:** Interface Amigável – interface simples e intuitiva para diferentes perfis de usuário;
- **NF003:** Banco de Dados MySQL – armazenamento e recuperação de dados em MySQL;
- **NF004:** Agilidade nas Operações – execução das operações no menor tempo possível;
- **NF005:** Manutenibilidade – suporte a manutenções sem interrupção do funcionamento do sistema.

3. Proposta Arquitetural

A arquitetura adotada, exibida na Figura 1, segue, como estilo arquitetural, o MVC (*Model-View-Controller*), onde a aplicação *web* é estruturada em três camadas bem definidas e com responsabilidades separadas. Os principais componentes incluem:

- *View (front-end)* – disponibiliza a interface de interação para os usuários do sistema, implementada com componentes *web* padrão como campos de texto, botões e *combo-boxes*;
- *Controller* – intermedia as requisições dos usuários recebidas pela *View*, acionando a lógica de negócio adequada e retornando a resposta correspondente;
- *Model (back-end)* – contém as entidades do sistema, as regras de negócio e a lógica de acesso aos dados;
- Banco de dados – registra e disponibiliza os dados por meio do *Model*, utilizando o banco de dados MySQL.

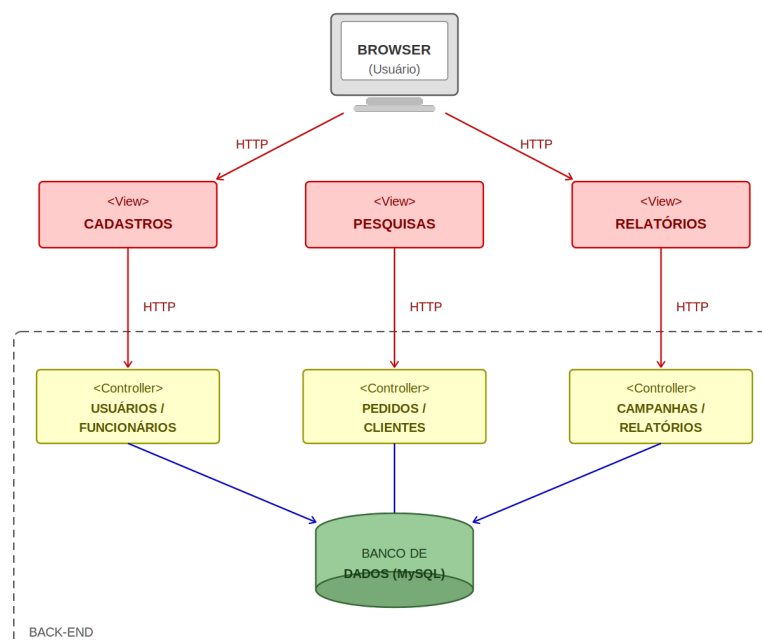


Figura 1. Visão geral da arquitetura do sistema

4. Decisões Arquiteturais

Agora, considerando os requisitos apresentados na Seção 2, abordamos as decisões arquiteturais que devem ser implementadas. Cada uma das sub-seções seguintes trata de uma decisão (uma decisão por cada ADR).

4.1. ADR-01: [Adoção do Estilo Arquitetural *Model-View-Controller*(MVC)]

Contexto:

Necessidade de organizar um sistema *web* com múltiplos módulos CRUD e diferentes perfis de usuário, separando claramente as responsabilidades de interface, controle e dados.

Decisão:

Adotar o estilo arquitetural MVC (*Model-View-Controller*), separando a aplicação em camadas de apresentação (*View*), controle (*Controller*) e lógica de negócio e dados (*Model*).

Consequências:

- Separação clara de responsabilidades entre as camadas;
- Facilidade de manutenção e evolução do sistema;
- Complexidade inicial maior na organização do projeto.

4.2. ADR-02: [Uso de Banco de Dados MySQL]

Contexto:

Necessidade de persistir dados de múltiplas entidades inter-relacionadas como funcionários, clientes, produtos, pedidos, equipes, embalagens e campanhas publicitárias.

Decisão:

Utilizar o MySQL como sistema gerenciador de banco de dados relacional para armazenamento e recuperação de todos os dados do sistema.

Consequências:

- solução gratuita e *open-source*¹;
- Boa performance para o volume de dados esperado;
- Requer administração especializada em SQL.

4.3. ADR-03: [Gerenciamento de Pedidos com Validação de Dependências]

Contexto:

Um pedido depende obrigatoriamente de produto, vendedor e cliente previamente cadastrados, exigindo validações para garantir integridade dos dados.

Decisão:

Implementar validação de dependências no *Controller* antes de persistir os dados, reforçada por chaves estrangeiras no banco de dados MySQL.

Consequências:

¹Refere-se à versão MySQL *Community Edition* que é gratuita e de código aberto. Existe também a versão MySQL *Enterprise Edition*, que é proprietária e paga.

- Consistência e integridade dos dados de venda garantidas;
- Viabiliza os relatórios gerenciais previstos nos requisitos funcionais;
- Em cenários de migração ou importação de dados, a exigência de integridade referencial pode dificultar a inserção de registros históricos.

4.4. ADR-04: [Geração de Relatórios Gerenciais por Período]

Contexto:

O sistema deve consolidar dados de múltiplos módulos para gerar relatórios gerenciais filtráveis por período, exigindo consultas complexas ao banco de dados.

Decisão:

Implementar a geração de relatórios no *Model* com consultas SQL otimizadas, renderizando os resultados diretamente no *browser* via *View*.

Consequências:

- Centralização da lógica de relatórios facilita manutenção;
- Consultas complexas podem degradar desempenho em grandes volumes.

4.5. ADR-05: [Gerenciamento de Campanhas Publicitárias]

Contexto:

O módulo de campanhas armazena dados financeiros sensíveis como valor previsto, valor de retorno e percentual de aumento de vendas, além de datas de início e término.

Decisão:

Implementar validações no *Controller* para datas e valores financeiros, vinculando cada campanha a um produto existente no sistema.

Consequências:

- Validações evitam cadastros inconsistentes;
- Vínculo com produtos permite análises cruzadas com vendas;
- Obrigatoriedade do vínculo limita campanhas institucionais.

4.6. ADR-06: [Interface Web com Componentes Padrão e Perfis de Acesso]

Contexto:

O sistema é acessado por diferentes perfis de usuário (Administrador, Gerente Geral, Gerente de Unidade e Atendente), cada um com permissões distintas.

Decisão:

Implementar a *View* com componentes HTML/CSS/JavaScript padrão, com o *Controller* verificando o perfil autenticado e disponibilizando apenas as funcionalidades permitidas.

Consequências:

- Compatibilidade com diferentes navegadores garantida;
- Controle de acesso por perfil aumenta a segurança;
- Manutenção de múltiplas *views* aumenta o volume de código.

4.7. ADR-07: [Estratégia de Manutenibilidade e Desempenho]

Contexto:

O sistema deve executar operações de forma ágil e suportar manutenções sem interrupção do serviço, conforme exigido pelos requisitos NF004 e NF005.

Decisão:

Adotar índices nos campos de busca frequente no MySQL e estruturar o sistema de forma modular no MVC, permitindo atualizações por módulo sem afetar os demais.

Consequências:

- Redução do tempo de resposta nas operações de pesquisa;
- Manutenções pontuais sem interrupção do sistema;
- Índices aumentam espaço em disco e impactam operações de escrita.

4.8. ADR-08: [Gerenciamento de Embalagens Vinculadas a Produtos]

Contexto:

Cada embalagem deve estar obrigatoriamente vinculada a um produto cadastrado, com dados de custo, volume e tipo relevantes para o controle de produção.

Decisão:

Implementar validação no *Controller* para garantir que o produto vinculado exista, utilizando chave estrangeira no banco para reforçar a integridade do vínculo.

Consequências:

- Integridade do vínculo produto-embalagem garantida;
- Validações previnem cadastros com dados inválidos;
- Ordem de cadastro obrigatória pode ser inconveniente operacionalmente.

5. Uso de Ferramentas de Apoio

No processo de definição da arquitetura do sistema Cervejaria BeboSim, foi utilizado o assistente de Inteligência Artificial Claude, desenvolvido pela Anthropic, como ferramenta de apoio.

A IA contribuiu nos seguintes aspectos:

- Consulta sobre estruturação e organização das seções do documento de arquitetura;
- Sugestão e discussão das decisões arquiteturais (ADRs), com base nos requisitos do sistema;
- Geração do diagrama visual da arquitetura MVC.

Limitações identificadas:

- As sugestões da IA precisaram ser revisadas e adaptadas pelo autor para garantir alinhamento com o documento de requisitos original.

6. Análise Crítica

A arquitetura MVC proposta atende aos requisitos principais do sistema BeboSim, promovendo separação clara de responsabilidades e facilitando o desenvolvimento e a manutenção do sistema. A escolha do MySQL como banco de dados relacional é adequada ao volume e à natureza dos dados gerenciados, que envolvem múltiplas entidades inter-relacionadas.

Entre os principais *trade-offs* identificados estão:

- Manutenibilidade vs. Complexidade inicial: a estrutura MVC exige mais organização no início do desenvolvimento, mas garante melhor manutenção a longo prazo;
- Desempenho vs. Integridade: as validações em múltiplas camadas aumentam a confiabilidade dos dados, mas podem adicionar latência nas operações de escrita;
- Flexibilidade vs. Consistência: os vínculos obrigatórios entre entidades garantem integridade referencial, mas impõem restrições de ordem de cadastro.

Melhorias futuras incluem:

- Implementação de paginação nos relatórios para melhorar o desempenho em bases com grande volume de registros;
- Adoção de mecanismo de cache para consultas frequentes, reduzindo a carga no banco de dados;
- Implementação de exclusão lógica para entidades vinculadas, preservando o histórico de pedidos e campanhas.

7. Conclusão

A arquitetura definida para o sistema Cervejaria BeboSim baseia-se no estilo MVC, adequado ao contexto de uma aplicação *web* de gestão com múltiplos módulos CRUD e diferentes perfis de usuário. As decisões arquiteturais registradas nas oito ADRs deste documento cobrem os principais requisitos funcionais e não-funcionais identificados no Documento de Requisitos de Software, equilibrando manutenibilidade, desempenho, usabilidade e integridade dos dados.

8. Ferramentas de Apoio

Nesta seção são disponibilizados os *links* para o código-fonte das ferramentas MCP desenvolvidas como suporte ao documento de arquitetura, bem como o vídeo de demonstração do seu funcionamento.

- **Código-fonte:** disponível no repositório GitHub em: <https://github.com/Nilmar-Pereira/arquitetura-cervejaria-bebosim.git>
- **Vídeo de demonstração:** disponível em: